

4. SQL Queries Demonstration

Quick Commerce System

4.1 Objective of SQL Queries

The objective of this section is to demonstrate the effective use of **SQL commands** for:

- Database definition
- Data manipulation
- Data retrieval
- Data analysis
- Enforcing constraints

As per the rubric, **at least 10 SQL queries** covering **minimum 5 functionalities** are demonstrated.

4.2 SQL Functionalities Covered

- ✓ DDL (CREATE, ALTER)
 - ✓ DML (INSERT, UPDATE, DELETE)
 - ✓ Aggregate Functions
 - ✓ GROUP BY & HAVING
 - ✓ Joins
 - ✓ Subqueries
 - ✓ Constraints
-

4.3 SQL Queries (SQL*Plus Compatible)

Query 1: Display all customers

(Simple *SELECT*)

```
SELECT * FROM customer;
```

Query 2: Display all available products

(WHERE condition)

```
SELECT product_name, price, stock_quantity
FROM product
WHERE availability_status = 'Available';
```

Query 3: Update stock quantity after an order

(DML – UPDATE)

```
UPDATE product
SET stock_quantity = stock_quantity - 2
WHERE product_id = 101;
```

Query 4: Delete a cancelled order

(DML – DELETE)

```
DELETE FROM orders
WHERE order_status = 'Cancelled';
```

Query 5: Find total number of customers

(Aggregate Function)

```
SELECT COUNT(*) AS total_customers
FROM customer;
```

Query 6: Find average price of products

(Aggregate Function)

```
SELECT AVG(price) AS average_price
FROM product;
```

Query 7: Display number of orders placed by each customer

(GROUP BY)

```
SELECT customer_id, COUNT(order_id) AS total_orders
FROM orders
GROUP BY customer_id;
```

Query 8: Display customers who placed more than one order

(GROUP BY + HAVING)

```
SELECT customer_id, COUNT(order_id) AS order_count
FROM orders
GROUP BY customer_id
HAVING COUNT(order_id) > 1;
```

Query 9: Display customer names along with their order details

(JOIN)

```
SELECT c.customer_name, o.order_id, o.order_status, o.total_amount
FROM customer c
JOIN orders o
ON c.customer_id = o.customer_id;
```

Query 10: Display order details with product names

(Multiple table JOIN)

```
SELECT o.order_id, p.product_name, oi.quantity, oi.price_at_order
FROM orders o
JOIN order_item oi ON o.order_id = oi.order_id
JOIN product p ON oi.product_id = p.product_id;
```

Query 11: Display products priced higher than average price

(Subquery)

```
SELECT product_name, price
FROM product
WHERE price > (SELECT AVG(price) FROM product);
```

Query 12: Add a constraint to ensure valid order status

(DDL – ALTER with CONSTRAINT)

```
ALTER TABLE orders
ADD CONSTRAINT chk_order_status
CHECK (order_status IN ('Pending', 'Delivered', 'Cancelled'));
```

4.4 Output Verification

- All queries execute successfully in **SQL*Plus**
 - Correct and meaningful outputs are generated
 - Queries reflect real-world quick commerce operations
 - Functionalities are clearly demonstrated
-